

DSA STUDY BLUEPRINT SERIES

45. Overview of Advanced Sorting

Overview of Advanced Sorting and Recursion Trees

Section 06 • C++ Core Data Structures & Algorithms

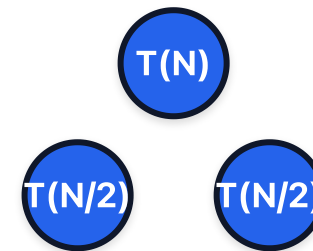
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Recursion Tree:** Visual representation of split depths and execution branches.
- **Analysis:** Summarizing recursion relations (e.g. Master Theorem).

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
// Conceptual recursion tree evaluation  
//  $T(N) = 2T(N/2) + O(N) \Rightarrow O(N \log N)$  (Merge Sort relation)
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(\text{Branching}^{\text{Depth}})$
Space Complexity	$O(\text{Depth})$ stack

Key Practices & Gotchas

- **Important:** Always draw recursion trees when evaluating unfamiliar recurrence relations.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run the partition splitting sequence for Merge Sort on list `[4, 2, 1, 3]` :

Depth Level	Divided segments	Sorting result	Merged result
1	[4, 2] & [1, 3]	Recursive sorting	[2, 4] & [1, 3] merged to [1, 2, 3, 4]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pivot heuristics:** Quick Sort speeds depend on pivot balance; choosing middle or random elements prevents $O(N^2)$ degradation.
- **Inversions:** Count array inversions during merge sorts to identify out-of-order bounds sizes.
- **Related LeetCode Problems:** #912 Sort an Array, #315 Count of Smaller Numbers After Self.